



课程目录

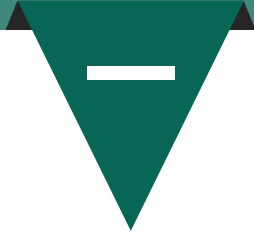
contents

一、LayUI简介

二、VUE 简介

三、AJAX 简介





LayUI&VUE



一、LayUI简介



LayUI概述

Layui 是一套免费的开源 Web UI 组件库，采用自身轻量级模块化规范，遵循原生态的 HTML/CSS/JavaScript 开发模式，极易上手，拿来即用。其风格简约轻盈，而内在雅致丰盈，甚至包括文档在内的每一处细节都经过精心雕琢，非常适合网页界面的快速构建；**Layui 是基于 jQuery 的框架，本身自带 jQuery**

谐音：类 UI	用途：用于更简单快速地构建网页界面
环境：全部主流 Web 浏览器（IE8 以下除外）	特性：原生态开发 / 轻量级模块化 / 外简内丰 / 开箱即用

官网 <https://layui.dev/> 下载LayUI 相关文件

一、LayUI简介



基本使用

```
<link rel="stylesheet" href="../css/layui.css">
```

```
<script src="../js/layui.js"></script>
```

```
请输入: <input type="text" value="默认值" id="layui_demo1_input"/>
```

```
<button id="layui_demo1_btn">点我</button>
```

```
<!-- 导入js相对应的模块jQuery.js, 将模块赋值给变量 -->
```

```
<script>
```

```
layui.use(['layer', 'form', 'jquery'], function() {  
    var layer = layui.layer  
    , $ = layui.jquery  
    , form = layui.form;  
    $("#layui_demo1_btn").click(function() {  
        var layui_demo1_input = $("#layui_demo1_input").val();  
        layer.msg(layui_demo1_input);  
    });  
    layer.msg('Hello World');  
});
```

```
});
```

```
</script>
```

layui.use() 函数用于模块加载

layui.use([mods], callback)

mods: 如果填写(选填), 必须是一个 layui 合法的模块名(不能包含目录)。

从 layui 2.6 开始, 若 mods 不填, 只传一个 callback 参数, 则表示引用所有内置模块。

callback: 即为模块加载完毕的回调函数。

从 layui 2.6 开始, 该回调会在 html 文档加载完毕后再执行, 确保你的代码在任何地方都能对元素进行操作。

一、LayUI简介



进阶应用

1、编写mymod.js

```
layui.define(function(exports) { //提示：模块也可以依赖其它模块，如：
  layui.define('mod1', callback);
  var obj = {
    hello: function(str) {
      alert('Hello ' + (str || 'mymod'));
    },
    say: function(str) {
      alert('say ' + (str || 'mymod'));
    }
  };

  //输出 mymod 接口
  exports('mymod', obj);
});
```

一、LayUI简介



进阶应用

2、编写extmodule.js

```
//配置自己定义的js文件
//config的设置是全局的
layui.config({
  base: 'js/' //假设这是你存放拓展模块的根目录
}).extend({ //设定模块别名
  mymod: 'mymod' //如果 mymod.js 是在根目录，也可以不用设定别名
});
```

3、html中引入js文件

```
<link rel="stylesheet" href="../../css/layui.css">
<script src="../../js/layui.js"></script>
<!-- 引入我们自己建立的js文件 -->
<script src="../../js/mymod.js"></script>
<script src="../../js/extmodule.js"></script>
```

一、LayUI简介



进阶应用

2、编写html

```
请输入: <input type="text" value="默认值" id="layui_demo1_input"/>
<button id="layui_demo1_btn">点我</button>
请输入: <input type="text" value="默认值" id="layui_demo2_input"/>
<button id="layui_demo2_btn">点我</button>
<script>
layui.use(['layer', 'form', 'jquery', 'mymod'], function() {
    var layer = layui.layer
    , $ = layui.jquery
    , mymod = layui.mymod
    , form = layui.form;
    $("#layui_demo1_btn").click(function() {
        var layui_demo1_input = $("#layui_demo1_input").val();
        layer.msg(layui_demo1_input);
    });

    $("#layui_demo2_btn").click(function() {
        var layui_demo2_input = $("#layui_demo2_input").val();
        mymod.say(layui_demo2_input);
        mymod.hello(layui_demo2_input);
    });

    layer.msg('Hello World');
});
</script>
```

一、LayUI简介



调用后台

```
<script type="text/javascript">
    layui.use(['jquery', 'layer'], function() {
        let $ = layui.jquery
        layer = layui.layer;
        $("#login").click(function() {
            $.ajax({
                url: "${pageContext.request.contextPath}/user.action?methodName=login",
                dataType: 'json', /* 格式 */
                data: { /* 要传输的值 */
                    username: $("#username").val(),
                    password: $("#password").val()
                }, success: function(data) { /* 成功的回掉函数 */
                    /* console.log(data); */
                    if(data.code == 200) {
                        layer.alert(data.msg, {icon: 1});
                        location.href='main.jsp';
                    } else{
                        layer.alert(data.msg, {icon: 2});
                    }
                }
            })
        })
    })
</script>
```


二、VUE简介



概述

Vue是一款用于构建用户界面的 JavaScript 框架。它基于标准 HTML、CSS 和 JavaScript 构建，并提供了一套声明式的、组件化的编程模型，帮助你高效地开发用户界面。无论是简单还是复杂的界面，Vue 都可以胜任。

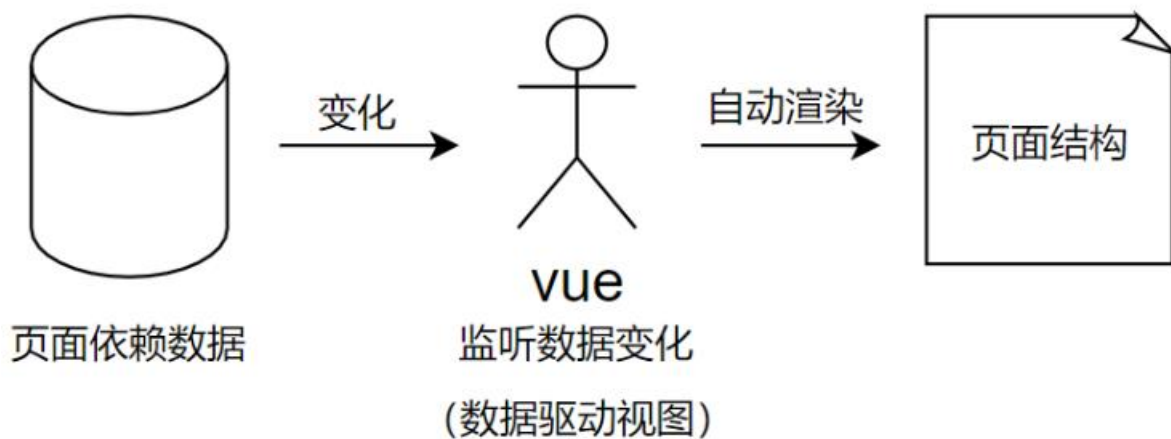
Vue.js是数据驱动的，你无需手动操作DOM。它通过一些特殊的HTML语法，将DOM和数据绑定起来。一旦你创建了绑定，DOM将和数据保持同步，每当变更了数据，DOM也会相应地更新

二、VUE简介



特性--数据驱动视图

数据的变化会 驱动 视图自动更新，这样一来，程序员只需要把数据维护好，至于页面的结构则会自动被 Vue 渲染出来。



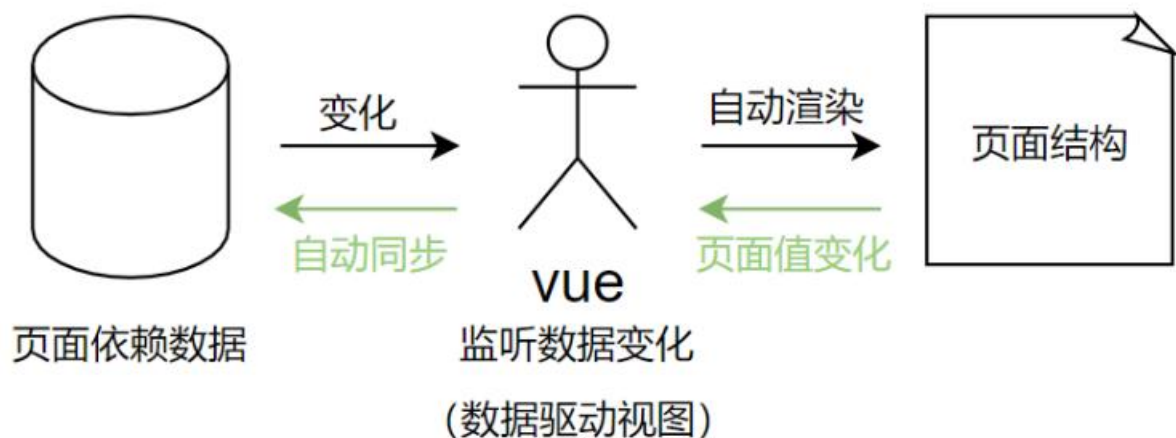
理解页面依赖的数据，通常页面依赖的数据是从后端查数据库来的，这里的页面依赖数据也是从后端来的，可能这个数据被放到一个 js 对象中，当这个 js 对象发生改变了，vue 就会监听到数据变化，也就会把新的数据渲染到页面上。

二、VUE简介



特性--双向数据绑定

在填写表单时，双向数据绑定可以辅助开发者在不操作 DOM 的前提下，自动把用户填写的内容同步到数据源



当用户填的表单数据发生变化，Vue 就会自动同步到 js 中，此时程序员就不用去操作 DOM 树来获取表单元素的最新值了。

二、VUE简介



案例01

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<!-- 引入VUE js库 -->
<!-- <script src="https://cdn.jsdelivr.net/npm/vue@2"></script> -->
<script src='../js/vue@2.js'></script>
</head>
<body>
<div id='div01'>
  {{msg}} <h2>{{blogName}}</h2>
</div>

</body>
<script>
  const app = new Vue({
    el:'#div01',//配置选择器
    data:{ //绑定数据
      msg: 'Hello vue',
      blogName: '我们去踢足球'
    }
  })
</script>
</html>
```

Hello vue

我们去踢足球

二、VUE简介



插值表达式

插值表达式是 Vue 的模板语法，也就是上述代码中的 `{{ }}`

重点：利用表达式进行插值，渲染到页面中

例：

```
<p>{{ age >= 18 ? '成年人' : "未成年" }}</p>
```

```
<p>{{ person.name }}</p>
```

```
<p>{{ name.toUpperCase() }}</p>
```

注意：

- 使用的数据必须存在(在 Vue 构造方法配置项 `data` 中声明)

- 支持的是表达式，并非语句，不支持 `if for` 等

- 不能在标签属性中使用 `{{ }}` 插值，如：

```
<a href="{{ url }}"></a>
```

```
<p title="{{ blog-title }}"></p>
```

```
<div id='div01'>
  {{msg}}
  <h2>{{blogName}}</h2>
  <p>{{ age >= 18 ? '成年人' : "未成年" }}</p>
</div>

</body>
<script>
  const app = new Vue({
    el: '#div01', //配置选择器
    data: { //绑定数据
      msg: 'Hello vue',
      blogName: '我们去踢足球',
      age: 20
    }
  })
```

二、VUE简介



VUE响应式

```
const app = new Vue({  
  el: '#div01', //配置选择器  
  data: { //绑定数据  
    msg: 'Hello vue',  
    blogName: '我们去踢足球',  
    age: 20  
  }  
})
```

```
<input type='button' value='test' onclick="fun01()"></input>  
</body>  
<script>  
  const app = new Vue({  
    el: '#div01', //配置选择器  
    data: { //绑定数据  
      msg: 'Hello vue',  
      blogName: '我们去踢足球',  
      age: 20  
    }  
  })  
  
  function fun01() {  
    app.msg = '你好 VUE';  
  }  
</script>
```

当页面依赖数据发生变化，Vue 则会监听到并自动渲染页面，此时如果将 msg 里的数据进行修改，Vue 也会自动渲染新的数据到页面上！

通过修改Vue实例名.属性名 即可修改data中的数据。

如：app.msg = "你好 VUE"

二、VUE简介



VUE方法

```
<div id='div01'>
  {{msg}}
  <h2>{{blogName}}</h2>
  <p>{{ age >= 18 ? '成年人' : "未成年" }}</p>
  <h3>count: {{count}}</h3>
  <input type='button' value='test' @click="fun01()"></input>
</div>

</body>
<script>
  let extData={
    msg: 'Hello vue',
    blogName: '我们去踢足球',
    age:20,
    count:0
  }
  const app = new Vue({
    el: '#div01', //配置选择器
    data: extData, //data属性指向Model, data: extData表示Model是extdata对象。
    methods: { //methods 用于定义的函数, 可以通过 return 来返回函数值
      fun01: function () {
        this.count++
      }
    }
  })
})
```

三、AJAX简介



AJAX 概述

异步(Asynchronous)的概念和同步相对(Synchronous)。当一个异步过程调用发出后，调用者不需要立刻得到结果，可以继续做自己的事情，等到过程调用完毕，再通过回调函数通知调用方。而同步情况下，调用方必须等待对方得到结果，才能继续运行

AJAX 包含以下五个部分：

- ✓ 异步数据获取技术，使用 XMLHttpRequest
- ✓ 基于标准的表示技术，使用 XHTML 与 CSS
- ✓ 动态显示和交互技术，使用 Document Object Model(文档对象模型)
- ✓ 数据互换和操作技术，使用 XML 与 XSLT
- ✓ JavaScript，将以上技术融合在一起

三、AJAX简介



AJAX 实例

```
function login(){  
    var account = document.getElementById("account").value;  
    var password = document.getElementById("password").value;  
    var xmlhttp=new XMLHttpRequest();  
    xmlhttp.open("GET", "webdemo01/jsp/ajax/info.jsp", true);
```

//2.响应结束后，处理函数设置

```
    xmlhttp.onreadystatechange=function() {  
/* 只有当一个完整的服务器响应已经收到了，函数才可以处理该响应。XMLHttpRequest 提供了readyState属性来对服务器响应进行判断。
```

```
    var resultDiv = document.getElementById("resultDiv")  
    if (xmlhttp.readyState==4) {  
        alert("xmlhttp.responseText==>" +xmlhttp.responseText);  
        }else{  
            alert("other===>" +xmlhttp.readyState);  
        } }  
    xmlhttp.send();           //发送请求  
}
```

三、AJAX简介



AJAX -API解释

步骤 1：在 IE 中实例化 Msxml2.XMLHTTP 对象：

IE浏览器使用 `new ActiveXObject("Msxml2.XMLHTTP");`

非IE浏览器使用： `new XMLHttpRequest();`

兼容 IE7+, Firefox, Chrome, Opera, Safari , Msxml2.XMLHTTP是IE浏览器内置的对象

步骤 2：指定异步提交的目标和提交方式，调用了 xmlhttp 的 open 方法：

`xmlHttp.open("get", "info.jsp", true);`

该方法一共 3 个参数：

参数 1—— 表示请求的方式，一般有如下选择：get、post。

参数 2—— 表示请求的目标是 info.jsp；当然，也可以在此处给 info.jsp 一些参数。如：`xmlHttp.open("GET", "info.jsp?account=0001", true);`

参数3——为true表示异步请求，否则表示非异步请求。异步请求可以通俗理解为后台提交，此种情况下，请求在后台执行。

三、AJAX简介



AJAX -API解释

步骤 3: 指定当 xmlhttp 状态改变时, 需要进行的处理。处理一般是以响应函数的形式进行:

```
xmlhttp.onreadystatechange=function() {  
    if (xmlhttp.readyState==4) {}  
    //处理代码  
}
```

常见的 readyState 属性值是:

- 0: 未初始化状态, 对象已创建, 尚未调用 open()
- 1: 已初始化状态, 调用 open()方法以后
- 2: 发送数据状态, 调用 send()方法以后
- 3: 数据传送中状态, 已经接到部分数据, 但接收尚未完成
- 4: 完成状态, 数据全部接收完成

三、AJAX简介



AJAX -API解释

步骤 4: 编写处理代码:

```
xmlHttp.onreadystatechange=function() {  
    if (xmlHttp.readyState==4) {  
        infoDiv.innerHTML = xmlHttp.responseText;  
    }  
}
```

步骤 5: 发出请求, 调用 xmlHttp 的 send 函数:

```
xmlHttp.send();
```

send()函数的参数要求跟在步骤二 xmlHttp.open(, ,); 第一个参数“请求方法”有关。

请求方法可以为get或post.

如果请求方式是 get 的话, send 可以没有参数, 或者参数为 null; 如果请求方式是 post, 可以将需要传送的内容传入 send 函数中以字符串的形式发出

三、AJAX简介



AJAX -API解释

步骤 4: 编写处理代码:

```
xmlHttp.onreadystatechange=function() {  
    if (xmlHttp.readyState==4) {  
        infoDiv.innerHTML = xmlHttp.responseText;  
    }  
}
```

步骤 5: 发出请求, 调用 xmlHttp 的 send 函数:

```
xmlHttp.send();
```

send()函数的参数要求跟在步骤二 xmlHttp.open(, ,); 第一个参数“请求方法”有关。

请求方法可以为get或post.

如果请求方式是 get 的话, send 可以没有参数, 或者参数为 null; 如果请求方式是 post, 可以将需要传送的内容传入 send 函数中以字符串的形式发出

三、AJAX简介



AJAX -优点

减轻服务器负担，避免整个浏览器窗口刷新时造成的重复请求
带来更好的用户体验
进一步促进页面呈现和数据本身的分离等



AJAX -缺点

对浏览器具有一定的限制，对于不兼容的浏览器，可能无法使用。
AJAX 没有刷新页面，浏览器上的"后退"按钮是失效的，因此，客户经常无法回退到以前的操作



HTTP状态码(status)

HTTP状态码(status)由三个十进制数字组成，第一个十进制数字定义了状态码的类型，后两个数字没有分类的作用。HTTP状态码共分为5种类型：

- **1xx**（临时响应）：表示临时响应并需要请求者继续执行操作的状态码。
- **2xx**（成功）：表示成功处理了请求的状态码。
- **3xx**（重定向）：表示要完成请求，需要进一步操作。通常，这些状态代码用来重定向。
- **4xx**（请求错误）：这些状态码表示请求可能出错，妨碍了服务器的处理。
- **5xx**（服务器错误）：这些状态码表示服务器在尝试处理请求时发生内部错误。这些错误可能是服务器本身的错误，而不是请求出错。

三、AJAX简介



HTTP 常见状态码(status)

- 200** 表示从客户端发来的请求在服务器端被正常处理了。
- 204** 表示请求处理成功，但没有资源返回。
- 301** 表示永久性重定向。该状态码表示请求的资源已被分配了新的URI，以后应使用资源现在所指的URI。
- 302** 表示临时性重定向。
- 400** 表示请求报文中存在语法错误。当错误发生时，需修改请求的内容后再次发送请求。
- 401** 表示未授权（Unauthorized），当前请求需要用户验证
- 403** 表示对请求资源的访问被服务器拒绝了
- 404** 表示服务器上无法找到请求的资源。除此之外，也可以在服务器端拒绝请求且不想说明理由时使用。
- 500** 表示服务器端在执行请求时发生了错误。也有可能是Web应用存在的bug或某些临时的故障。
- 503** 表示服务器暂时处于超负载或正在进行停机维护，现在无法处理请求。

新的开始，新的起点，让我们一起为梦想而努力。

谢谢聆听！

